KIẾN TRÚC VÀ LUỒNG HÀM
POC ĐÁNH GIÁ CÂU TRẢ LỜI

Mục tiêu: đối chiếu phần thể hiện trong một câu trả lời với các tiêu chí và thang điểm đã khai báo; tạo phản hồi có căn cứ. Đầu vào gồm Question, các Rubric và Answer.originalText. Đầu ra gồm kết quả từng năng lực, bằng chứng, lý do và Feedback.

Tài liệu mô tả mã triển khai hiện tại. Tên file và hàm giữ nguyên tiếng Anh để đối chiếu trực tiếp trong IDE. Không bao gồm kết quả chạy thử.

1. Cách tổ chức theo MVC-S

Vị trí	Trách nhiệm
source/main.py	Điểm khởi chạy duy nhất. Ghép Repository, Service, Controller, View và bộ diễn giải. Không chứa quy tắc chấm.
source/Model/	Cấu trúc dữ liệu và ràng buộc trường. Model/repositories chứa giao diện đọc/lưu và cách cài bằng JSON.
source/service/	Kiểm tra rubric, diễn giải câu trả lời, kiểm tra bằng chứng, tính điểm, tạo phản hồi và điều phối quy trình.
source/Control/	Nhận hành động từ View, kiểm tra request, gọi Service và đổi lỗi dự kiến thành phản hồi có cấu trúc.
source/view/	Nhận tham số terminal và trình bày kết quả. Không trực tiếp đọc dữ liệu hoặc gọi AI.
source/reports/	PDF kiến trúc. results/ chỉ được tạo khi ứng dụng lưu dữ liệu kết quả; các kết quả đó không được đưa vào tài liệu này.
test/; examples/evaluateAnswer/	Mã kiểm tra nằm trong test. examples chỉ giữ JSON, bài trả lời mẫu và hướng dẫn dữ liệu, không chứa một chương trình chấm thứ hai.

Theo AGENTS.md của repo, logic nghiệp vụ nằm trong **service**. Các file __init__.py là dấu mốc package của Python, không có nghiệp vụ. Ngoài main.py và dấu mốc package này, source không có file code đứng ngang hàng các thư mục kiến trúc.

2. Vì sao nhiều file có chữ evaluation?

Chúng dùng chung tên tính năng nhưng có trách nhiệm khác nhau: evaluationData mô tả dữ liệu, evaluationController nhận yêu cầu, evaluationService điều phối và evaluationView trình bày. Mỗi loại quy tắc chấm chỉ có một nơi cài đặt.

Các phần đã gộp: entry phụ evaluateAnswer.py vào main.py; các thao tác trạng thái trong answerEvaluation.py vào evaluationService.py; CLI và định dạng text vào evaluationView.py; cấu hình Gemini vào geminiInterpreter.py. saveReport hiện chứa trực tiếp thao tác ghi, không còn một hàm chuyển tiếp chỉ để gọi writeNewReport.

EvaluationRepository và EvidenceInterpreter là Protocol: chỉ khai báo thao tác cần có. JsonEvaluationRepository, GeminiInterpreter và RecordedInterpreter cài các thao tác đó. Trùng tên phương thức tại interface và implementation là cách thay thế phụ thuộc, không phải hai thuật toán chấm.

3. Luồng gọi hàm đúng theo thứ tự chạy

Lệnh khởi chạy gọi **main.main()**. Sơ đồ dưới đây ghi rõ các nhánh và hàm trung gian; mũi tên trở về được lược để dễ đọc.

```
main(arguments)
  createController()
    JsonEvaluationRepository(resultDirectory)
    EvaluationService(repository, createInterpreter)
    EvaluationController(service)
  readArguments(arguments)
  runView(controller, args)
    controller.submitAnswer(requestData)
      EvaluationRequest.model_validate(requestData)
        checkRecording()
      service.evaluateFiles(...)
        repository.resolveOutput(outputPath)
        loadInputs(inputPath, answerPath)
          repository.loadInput() -> readTextFile()
          [new answer] repository.loadAnswer() -> readTextFile()
          [new answer] EvaluationInput.model_validate(...)
        [recorded] repository.loadRecording() -> readTextFile()
        createInterpreter(providerName, recording)
          recorded -> RecordedInterpreter(recording)
          gemini   -> readGeminiSettings() -> GeminiInterpreter(...)
        assessAnswer(inputs, interpreter)
          createEvaluation(...) -> validateInputs()
            requireUnique(); validateLevels() -> findLevel()
          evaluateCriteria(evaluation, interpreter)
            validateInputs()
            interpreter.interpret(inputs)
            validateInterpretation(inputs, interpretation)
              validateCheck() -> validateQuote() / validateOmission()
            [each rubric] scoreChecks() -> findLevel() twice
            EvaluationResult(...)
          generateFeedback(evaluation)
            validateResults()
            [each result] buildFeedback(result, rubric)
          readReport(evaluation, interpreter)
            validateResults(); inputDigest(); EvaluationReport(...)
        repository.saveReport(report, resultPath)
      EvaluationResponse(report, outputPath) -> checkOutcome()
    renderReport(report) OR report.model_dump_json()
    print result; print saved path; return exit code
```

Thứ tự bảo vệ: kiểm tra request và vị trí output trước; xác thực cấu trúc dữ liệu và rubric trước khi gọi AI; xác minh diễn giải trước khi tính điểm; hoàn tất phản hồi trước khi lưu. Nếu một bước lỗi, không xuất một báo cáo chấm dở dang.

4. Trích xuất và làm rõ diễn ra ở đâu?

GeminiInterpreter.interpret() gọi buildRequest(), trong đó systemPrompt() và promptPayload() tạo nội dung gửi AI. Sau phản hồi, parseResponse() trả Interpretation gồm một review/rubric và một check/criterion. Việc chọn nội dung quan trọng, hiểu ý và nhận diện mơ hồ được thực hiện trong lần diễn giải này, không có một bộ trích xuất từ khóa riêng.

Làm rõ trong PoC: phần mơ hồ được gắn ambiguity và insufficientEvidence; observation nêu điều chưa thể kết luận. scoreChecks() liệt kê tiêu chí cần làm rõ nếu chúng làm thay đổi mức điểm. Hiện chưa có hàm hỏi lại người dùng hay vòng hội thoại bổ sung. RecordedInterpreter chỉ phát lại diễn giải đúng inputHash, không trích xuất thông tin mới.

5. Quy tắc chấm và các trường hợp dừng

Trường hợp	Cách xử lý thực tế
Thang điểm	scoreLevels chứa số mức thực tế qua độ dài danh sách, value, description, condition, requiredMet và minimumMet. findLevel chọn mức cao nhất thỏa các điều kiện máy đọc được; Python không suy ngưỡng từ câu mô tả.
Tiêu chí chưa rõ	scoreChecks tính mức chắc chắn từ met và mức cao nhất có thể từ met + insufficientEvidence. Hai mức khác nhau: score=null. Hai mức giống nhau: giữ điểm đó, vẫn ghi riêng tiêu chí chưa rõ. Điều này đúng vì quy tắc chỉ dùng điều kiện dương và ngưỡng số tiêu chí.
Sai hoặc thiếu	unmet có căn cứ sai hoặc phần bắt buộc bị thiếu. Chỉ cho phép kết luận từ omission khi criterion.omissionMeansUnmet=True. Không tự coi mọi phần thiếu là sai.
Trích dẫn	validateQuote kiểm tra nguyên văn và vị trí [startOffset:endOffset]. Nếu quote xuất hiện nhiều lần, bắt buộc cung cấp vị trí. Việc quote tồn tại chưa tự chứng minh nhận định ngữ nghĩa của AI là đúng.
Kiểm tra trạng thái	validateResults kiểm tra lại ID, liên kết, tiêu chí, điểm/lý do và Feedback. Các phép kiểm tra được gọi lại tại biên thao tác ADT để không tin trạng thái truyền vào; phần quy tắc vẫn chỉ có một định nghĩa.
Lỗi xử lý	Lỗi cấu trúc, rubric, căn cứ, mạng, timeout hoặc lưu trữ dừng quy trình. Controller trả mã lỗi dự kiến. ValueError bất ngờ của chương trình được để lộ traceback để debug, không đổi thành lỗi file.

6. Danh mục file và từng hàm trong mã ứng dụng

Các đường dẫn từ đây tính tương đối từ source/. Với lớp chỉ khai báo dữ liệu, bảng mô tả lớp thay vì liệt kê những constructor do thư viện tự sinh.

main.py

Ghép các thành phần cụ thể tại một nơi, để thay provider hoặc cách lưu mà không sửa thuật toán chấm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
createInterpreter(providerName, recording)	Trả EvidenceInterpreter. Recorded cần bản diễn giải; Gemini đọc cấu hình môi trường. Provider lạ báo providerConfig.
createController()	Tạo JsonEvaluationRepository với thư mục kết quả mặc định, truyền repository và factory vào EvaluationService, rồi tạo EvaluationController.
main(arguments)	Gọi createController, readArguments và runView. Trả số nguyên exit code; khối __main__ chuyển nó thành mã thoát chương trình.

Control/evaluationSchemas.py

Hợp đồng request/response tại ranh giới giao diện. Dữ liệu đầu vào không hợp lệ bị chặn trước khi chạy Service.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationRequest; checkRecording()	Giữ inputPath, providerName và các đường dẫn tùy chọn. checkRecording yêu cầu recordingPath khi và chỉ khi provider là recorded; trả chính request đã hợp lệ.
EvaluationFailure	Dữ liệu lỗi gồm code, message, retryable và fields. Không chứa điểm hoặc kết luận về người học.
EvaluationResponse; checkOutcome()	Chứa report + outputPath hoặc chỉ error. checkOutcome loại trạng thái thiếu một nửa kết quả hoặc vừa thành công vừa lỗi.

Control/evaluationController.py

Nhận hành động từ View, điều phối Service và chuẩn hóa lỗi dự kiến; không tự đọc file hoặc chấm điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationController.__init__(service)	Nhận Service qua tham số để có thể thay bằng đối tượng khác khi tích hợp hoặc kiểm tra.
submitAnswer(requestData)	Xác thực EvaluationRequest, gọi evaluateFiles và trả EvaluationResponse. Chuyển EvaluationError, ValidationError, FileExistsError, UnicodeError và OSError thành các mã lỗi tương ứng. Không bắt mọi ValueError.

view/evaluationView.py

Chứa toàn bộ giao diện terminal của PoC. Việc in không làm thay đổi dữ liệu hay tính điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
readArguments(arguments)	Nhận --input, --provider, --recording, --answerFile, --output, --format và trả argparse.Namespace.
runView(controller, args)	Gọi submitAnswer một lần, in lỗi ra stderr với exit code 1; nếu thành công, in text/JSON và đường dẫn lưu, trả 0.
renderReport(report)	Tạo chuỗi trình bày nguồn diễn giải, phạm vi, mức đạt, mức cho phép, feedback và trích dẫn. Nhận báo cáo hoàn chỉnh từ luồng đánh giá.

Model/answerData.py

Schema đầu vào dùng chung. Pydantic kiểm tra kiểu, giới hạn và cấm trường thừa; quy tắc liên kết nhiều đối tượng nằm ở Service.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
FrozenData	Lớp nền bất biến, extra=forbid và validate_default=True.
Question; Answer	Question giữ ID/version, đề, yêu cầu, ngữ cảnh và mức dự kiến. Answer giữ ID/revision, questionId và nguyên văn; không tự chuẩn hóa hoặc cắt nội dung trả lời.
Criterion	Mô tả tiêu chí, acceptanceRule, reviewRule và omissionMeansUnmet. Các trường này hướng dẫn diễn giải bằng chứng.
ScoreLevel; Rubric	ScoreLevel giữ điểm, mô tả, điều kiện và ngưỡng máy đọc. Rubric nhóm tiêu chí và thang điểm theo một năng lực, tham chiếu câu hỏi.
EvaluationInput	Gói Question, tuple Rubric, Answer và cờ exampleOnly thành snapshot của lượt đánh giá.
Text, Score, Count, Version	Kiểu dữ liệu có giới hạn: văn bản có nội dung, điểm hữu hạn, đếm tiêu chí có giới hạn, phiên bản nguyên dương.

Model/evaluationData.py

Schema diễn giải, trạng thái và kết quả. Các trạng thái chưa đủ căn cứ được biểu diễn rõ để không lẫn với điểm 0.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
Evidence; CriterionCheck	Evidence giữ loại, quote, vị trí, observation và offsets. CriterionCheck giữ criterionId, status và một hay nhiều Evidence.
RubricReview; Interpretation	Gom các check theo rubric và gom tất cả review thành đầu ra của interpreter.
EvaluationResult; checkScore()	Kết quả một rubric với resultId, answerId, score, status, checks và reason. checkScore yêu cầu graded có điểm và insufficientEvidence có score=null.
Feedback; AnswerEvaluation	Feedback liên kết resultId với giải thích và các điểm chưa đạt. AnswerEvaluation giữ snapshot đầu vào cùng results/feedbackItems trung gian.
EvaluationReport; RecordedReview	Report giữ inputs, inputHash, provider/model/prompt và kết quả. RecordedReview giữ inputHash với interpretation để phát lại đúng dữ liệu.

service/evaluationService.py

Một file điều phối toàn bộ đánh giá. Các hàm độc lập thao tác trên dữ liệu trong bộ nhớ; lớp EvaluationService nối quy trình đó với repository.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationService.__init__(repository, interpreterFactory)	Giữ các phụ thuộc qua interface; không tạo provider hoặc kho lưu cụ thể trong nghiệp vụ.
loadInputs(inputPath, answerPath)	Đọc EvaluationInput. Nếu có answerPath, giữ nguyên văn file mới, tạo answerId mới/revision=1 và xác thực lại gói dữ liệu. Trả EvaluationInput mới; không ghi đè đầu vào.
evaluateFiles(...)	Kiểm tra lựa chọn provider, chọn nơi lưu, đọc input/recording, tạo interpreter, gọi assessAnswer, lưu báo cáo hoàn chỉnh và trả (report, resultPath).
assessAnswer(inputs, interpreter)	Hàm quy trình trong bộ nhớ: createEvaluation -> evaluateCriteria -> generateFeedback -> readReport. Không đọc/ghi file.
createEvaluation(question, rubrics, answer, exampleOnly)	Tạo EvaluationInput, gọi validateInputs, trả AnswerEvaluation chưa có results/feedback.
evaluateCriteria(evaluation, interpreter)	Chỉ nhận trạng thái chưa chấm. Lấy diễn giải, xác minh, gọi scoreChecks cho từng rubric, tạo EvaluationResult và trả bản trạng thái mới.
generateFeedback(evaluation)	Yêu cầu đã có results và chưa có feedback. Gọi validateResults rồi buildFeedback cho mỗi result; trả trạng thái mới.
validateResults(evaluation)	Kiểm tra ID duy nhất, đủ review/check, answerId đúng, điểm/lý do khớp scoreChecks và feedback khớp buildFeedback. Không trả kết quả; báo lỗi nếu sai.
readReport(evaluation, interpreter)	Kiểm tra trạng thái, thêm inputHash và metadata provider, trả EvaluationReport. ADT có thể đọc trạng thái mới tạo; luồng evaluateFiles chỉ lưu sau khi đã hoàn tất feedback.

service/rubricRules.py

Nơi duy nhất định nghĩa cách xác thực rubric và ánh xạ các criterionCheck sang mức điểm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
requireUnique(values, label)	So số phần tử với tập hợp; phát hiện trùng rubricId, criterionId, competency hoặc requiredMet.
validateInputs(inputs)	Kiểm tra answer và rubric thuộc đúng question, ID/năng lực không lặp; gọi validateLevels cho mỗi rubric.
validateLevels(rubric)	Yêu cầu điểm khác nhau tăng dần, mức nền không đòi tiêu chí, tham chiếu/ngưỡng hợp lệ. Duyệt các tập tiêu chí để bảo đảm từng mức có thể đạt; số tiêu chí tối đa là 12.
findLevel(rubric, metIds)	Duyệt mức từ cao xuống thấp; trả mức đầu tiên có requiredMet là tập con của metIds và đủ minimumMet.
scoreChecks(rubric, checks)	Tính hai biên bằng findLevel. Trả (score, reason); score=null nếu phần chưa rõ có thể thay đổi mức. Không cộng điểm các năng lực.

service/evidenceReview.py

Kiểm tra hình dạng và liên kết của bằng chứng trước khi dùng cho điểm; giữ căn cứ có thể truy về nguyên văn.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
invalidEvidence(message)	Phát sinh EvaluationError có code invalidEvidence để các nhánh xác thực dùng cùng cơ chế lỗi.
validateQuote(evidence, originalText)	Kiểm tra quote nguyên văn, đủ hai offsets, khoảng vị trí hợp lệ. Nếu quote duy nhất và thiếu cả hai offsets, tự xác định vị trí. Trả Evidence có location chuẩn hóa.
validateOmission(evidence, criterion, check)	Yêu cầu quote/offsets null, không dùng omission để chứng minh met, chỉ kết luận unmet khi rubric cho phép. Trả bằng chứng ghi phạm vi toàn bài.
validateCheck(check, criterion, originalText)	Kiểm tra từng Evidence, buộc ambiguity đi với insufficientEvidence và met phải có excerpt. Trả check đã chuẩn hóa.
validateInterpretation(inputs, interpretation)	Buộc đúng một review mỗi rubric, một check mỗi criterion, không ID lạ/trùng/thiếu. Gọi validateCheck và sắp theo thứ tự rubric đầu vào.

service/feedbackBuilder.py

Tạo nội dung phản hồi từ kết quả đã xác minh để điểm và giải thích có cùng căn cứ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
buildFeedback(result, rubric)	Ghép reason và observations theo từng criterion; tạo identifiedIssues cho unmet với acceptanceRule tương ứng. Trả Feedback. Không thêm một lần chấm AI.

service/evaluationErrors.py

Kiểu lỗi nghiệp vụ dùng giữa Service và Controller; giúp lỗi xử lý không biến thành kết luận về bài làm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationError.__init__(code, message, retryable)	Giữ mã lỗi, nội dung tiếng Anh và cờ có thể thử lại. Các chỗ bắt lỗi mạng dùng raise ... from ... để giữ nguyên nhân khi debug.

service/providers/evidenceInterpreter.py

Định nghĩa giao diện nguồn diễn giải và cách ràng buộc input với bản diễn giải ghi sẵn.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvidenceInterpreter.interpret(inputs)	Protocol nhận EvaluationInput và trả Interpretation; đồng thời cung cấp providerName, modelName, promptVersion. Không tự cài việc chấm.
inputDigest(inputs)	Chuẩn hóa JSON bằng thứ tự key cố định rồi tính SHA-256. Thay câu hỏi, rubric, bài hoặc metadata đầu vào sẽ đổi hash.

service/providers/recordedInterpreter.py

Nguồn diễn giải có sẵn cho PoC; thay Gemini thông qua cùng interface.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
RecordedInterpreter.__init__(recording)	Nhận RecordedReview đã xác thực cấu trúc.
interpret(inputs)	So recording.inputHash với inputDigest(inputs). Khớp thì trả interpretation; khác thì báo recordingMismatch. Không gọi mạng hoặc tính điểm.

service/providers/geminiInterpreter.py

Cô lập cấu hình và giao tiếp với dịch vụ AI; Service lõi chỉ nhận Interpretation có cấu trúc.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
GeminiSettings; readGeminiSettings()	Lấy GEMINI_API_KEY, GEMINI_MODEL, EVALUATION_TIMEOUT. API key không xuất hiện trong repr của settings; timeout được đọc thành số giây.
GeminiInterpreter.__init__(apiKey, modelName, timeoutSeconds)	Kiểm tra key, tên model và timeout 1-120 giây; lưu cấu hình cho các request.
buildRequest(inputs)	Gọi systemPrompt và promptPayload, tạo request có JSON schema của Interpretation, giới hạn body 250 KB và đưa API key vào header. Trả urllib Request.
interpret(inputs)	Gọi buildRequest rồi urlopen một lần; đọc tối đa 1 MB và gọi parseResponse. Phân biệt quota, HTTP, timeout, mạng; không tự thử lại hay tạo điểm dự phòng.
parseResponse(rawBody)	Yêu cầu một candidate hoàn chỉnh STOP, không bị block. Ghép text ngoài phần thought và xác thực Interpretation; JSON/shape sai trả providerOutput.

service/prompts/evaluationPrompt.py

Giữ hướng dẫn diễn giải ở một chỗ để xem xét cách AI xử lý yêu cầu, lời giải tương đương và thông tin mơ hồ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
systemPrompt()	Trả hướng dẫn tiếng Anh: xem bài làm là dữ liệu, không làm theo chỉ dẫn trong bài, không bịa lập luận/trích dẫn và không cho điểm; chỉ nhận xét theo criterion.
promptPayload(inputs)	Trả dict chứa question, rubricId/competency/criteria và answer. Không gửi scoreLevels cho AI vì điểm do Python tính.
promptVersion	Nhãn phiên bản prompt được lưu trong báo cáo để truy xuất cách diễn giải đã sử dụng.

Model/repositories/evaluationRepository.py

Giao diện persistence giúp EvaluationService dùng file, bộ nhớ hoặc cơ sở dữ liệu mà không đổi các bước chấm.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
EvaluationRepository.loadInput(inputPath)	Hợp đồng đọc và trả EvaluationInput.
loadAnswer(answerPath)	Hợp đồng trả nguyên văn câu trả lời kiểu str.
loadRecording(recordingPath)	Hợp đồng đọc và trả RecordedReview.
resolveOutput(outputPath)	Hợp đồng chọn và kiểm tra vị trí lưu; trả Path.
saveReport(report, outputPath)	Hợp đồng lưu báo cáo hoàn chỉnh, không có giá trị trả về.

Model/repositories/jsonEvaluationRepository.py

Cài interface bằng UTF-8/JSON. Mọi thao tác file của PoC tập trung ở đây.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
readTextFile(filePath)	Đọc tối đa 1 MB + 1 byte để phát hiện file quá lớn; giải mã utf-8-sig. Không biến đổi xuống dòng. File quá lớn báo OSError, sai mã hóa báo UnicodeError.
JsonEvaluationRepository.__init__(reportDirectory)	Giữ thư mục mặc định cho kết quả khi không có --output.
loadInput(inputPath)	Gọi readTextFile rồi EvaluationInput.model_validate_json.
loadAnswer(answerPath)	Gọi readTextFile và trả nguyên văn; không gọi AI.
loadRecording(recordingPath)	Gọi readTextFile rồi RecordedReview.model_validate_json.
resolveOutput(outputPath)	Dùng đường dẫn chỉ định hoặc tạo report-UUID.json; từ chối file đã tồn tại trước khi chấm.
saveReport(report, outputPath)	Ghi JSON vào file tạm cùng thư mục, flush/fsync, tạo file đích bằng hard link và dọn file tạm. Nếu đích đã xuất hiện thì thất bại, không ghi đè. Cơ chế này cần filesystem hỗ trợ hard link.

7. Các Model dùng chung có trước PoC

Model/ids.py: các NewType như QuestionId, RubricId, CriterionId, AnswerId và VersionedRef định danh bản ghi. PoC tái sử dụng các ID này.

Model/question.py: ReviewStatus, LicenseStatus, ReviewChecks, ReviewMetadata, SourceMetadata, Skill, CriterionLevel, Criterion, Rubric và Question mô tả ngân hàng câu hỏi theo hợp đồng P03.

Model/evaluation.py: EvaluationStatus, CriterionStatus, AnswerRevisionRef, Evidence, CriterionCheck và EvaluationResult mô tả kết quả theo P03. Các lớp là dataclass/enum, không có thuật toán chấm riêng.

P03 chấm theo criterion và tham chiếu một rubric/câu; PoC theo abstraction mới có nhiều rubric theo năng lực và mức điểm theo rubric. Vì ý nghĩa trường khác nhau, gộp hai schema bằng cách đổi tên sẽ làm sai hợp đồng dùng chung. Quyết định này được ghi trong docs/data_contract.md; cần adapter hoặc thống nhất hợp đồng khi tích hợp với tính năng khác.

8. File kiểm tra: nhiệm vụ của các hàm

Phần này chỉ giải thích mã kiểm tra; không chứa số liệu hoặc kết quả chạy thử.

test/answerEvaluationTest.py

Kiểm tra hành vi chấm và bảo toàn căn cứ.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
inputs; recording; startEvaluation; statusCheck	Tạo dữ liệu và trạng thái hỗ trợ kiểm tra; không là một luồng đánh giá của ứng dụng.
testCompleteFlow; testLifecycleGuards	Kiểm tra chuỗi ADT, liên kết feedback và các chuyển trạng thái hợp lệ.
testScoreCombinations; testCustomScoreValues; testCountThresholds	Kiểm tra các khả năng met/unmet/chưa rõ, mức điểm tùy chỉnh và ngưỡng số tiêu chí.
testInvalidRubrics; testInvalidInterpretations; testQuoteOffsets	Kiểm tra từ chối rubric/ID/bằng chứng sai, vị trí trích dẫn và trích dẫn lặp.
testUnknownKeepsNull; testEmptyAnswer; testSchemaRejectsCoercion	Kiểm tra null, bài rỗng và từ chối ép kiểu sai.
testAtomicProviderFailure; testLateFailureAtomicity	Không giữ kết quả chấm dở dang khi nguồn diễn giải hoặc bước cuối lỗi.
testRecordingInputBinding; testRejectsChangedResults	Không dùng recording sai hash hoặc trạng thái bị sửa điểm không theo rubric.

test/evaluationArchitectureTest.py

Kiểm tra ranh giới lớp và khả năng thay thế phụ thuộc.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
testLayerDependencies; testNoFileAccess; testCompositionRoot	Kiểm tra import theo MVC-S, View/Controller không đọc file và main ghép đúng các phụ thuộc.
testControllerRejectsShape; testProgrammingErrorStaysVisible	Request sai bị chặn; lỗi lập trình không bị đổi thành lỗi file.
MemoryRepository	Cài năm thao tác Repository bằng dữ liệu mẫu và danh sách lưu trong bộ nhớ; dùng để quan sát Service độc lập với file.
testRepositoryCanBeReplaced; testFailureDoesNotSave; testNewAnswerRevision	Thay kho lưu, không lưu khi lỗi và không sửa câu trả lời cũ khi nhận bài mới.
testConfigHidesSecret; testSingleRootEntryAndEnglishCode	API key không nằm trong repr; chỉ main và package marker ở cấp source; code source dùng tiếng Anh.

test/evaluationCommandTest.py

Kiểm tra đầu vào/đầu ra thực tế của lệnh chạy.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
tempPath; runCli	Tạo nơi ghi tạm và gọi chương trình CLI bằng Python hiện tại.
testCommandRoundTrip; testTextPresentation	Kiểm tra JSON in/lưu khớp nhau, không ghi đè và text có metadata/bằng chứng.
testNewAnswerNeedsReview; testInvalidFileStopsBeforeAssessment	Bài mới cần recording mới; sai UTF-8 hoặc quá dung lượng phải dừng trước khi tạo kết quả.

test/geminiInterpreterTest.py

Kiểm tra giao tiếp Gemini bằng phản hồi giả lập, không gọi mạng trong bộ kiểm tra.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
sampleInputs; sampleResponse; testStructuredRequest	Tạo dữ liệu giả lập, kiểm tra request có schema, đúng bài gốc và API key không nằm trong URL/body.
testHttpFailures; testConnectionFailures	Phân biệt quota, HTTP, timeout và lỗi mạng; không đưa dữ liệu lỗi bí mật ra thông báo.
testInvalidResponse; testRejectsProviderScore	Từ chối đầu ra dở dang/sai schema hoặc AI tự thêm điểm.
testMissingConfiguration; testOversizeResponse	Từ chối thiếu key/model hoặc phản hồi quá lớn.

test/conftest.py; test/test_environment.py

Hạ tầng smoke test web của repo, không tham gia đánh giá câu trả lời.

Hàm / lớp	Nhiệm vụ, đầu vào và đầu ra
QuestionPreview; smoke_client; preview	Tạo FastAPI nhỏ, schema request và route /preview để kiểm tra thư viện web và template.
testWebStackRendersUnicode	Kiểm tra nội dung có ký hiệu Unicode được giữ đúng khi render.
test_web_stack_rejects_missing_question	Kiểm tra request thiếu question trả lỗi validation.

9. OOP và giới hạn hiện tại

Nên dùng OOP có chọn lọc. Controller, Service, Repository và Interpreter có phụ thuộc/trạng thái rõ nên phù hợp với class. Các quy tắc rubric, bằng chứng và phản hồi là hàm nhận dữ liệu rồi trả kết quả, dễ đọc và debug. Không cần chuyển thêm toàn bộ các hàm này thành class chỉ để có OOP.

Giới hạn cần hiểu: các kiểm tra Python bảo đảm đúng schema, liên kết, trích dẫn và công thức điểm; chúng không tự chứng minh AI hiểu đúng ngữ nghĩa. Nội dung mẫu chưa phải bộ câu hỏi được duyệt. PoC không chạy code do người dùng nộp, chưa có hội thoại làm rõ và chưa suy ra năng lực tổng thể. Các điều này không được trình bày như tính năng đã hoàn thành.